

Отчет по лабораторной работе №6

Блокировки и мониторинг

Дата: 2025-12-15

Семестр: 4 курс 1 полугодие – 7 семестр

Группа: ПИЖ-6-о-22-1

Дисциплина: Администрирование баз данных

Студент: Гойдь Алина Яновна

Руководитель практики: Щеголев Алексей Алексеевич

старший преподаватель департамента цифровых, робототехнических систем и электроники института перспективной инженерии

Цель работы

Изучить систему блокировок в PostgreSQL и методы мониторинга активности сервера. Получить практические навыки анализа статистики, диагностики блокировок и взаимоблокировок, использования инструментов мониторинга.

Теоретическая часть

- **Блокировки (Locks):** Механизм обеспечения согласованности данных при параллельном доступе к одним и тем же объектам
- **Типы блокировок:** Объектные (таблицы, представления), строковые (отдельные записи), блокировки в памяти (buffer pins)
- **Мониторинг активности:** Отслеживание работы сервера через системные представления и функции
- **Взаимоблокировка (Deadlock):** Ситуация взаимного ожидания, когда две или более транзакции блокируют друг друга
- **Статистика использования:** Накопление данных о количестве обращений, изменений и производительности объектов БД

Ключевые термины

- **pg_locks:** Представление, показывающее все активные блокировки в системе
- **pg_stat_all_tables:** Статистика обращений к таблицам (INSERT, UPDATE, DELETE, SELECT)
- **pg_stat_statements:** Расширение для накопления статистики выполнения SQL-запросов
- **pg_buffercache:** Расширение для просмотра содержимого буферного кеша
- **AccessShareLock:** Самая слабая блокировка, захватывается при SELECT
- **RowExclusiveLock:** Блокировка при INSERT, UPDATE, DELETE
- **RowShareLock:** Блокировка при SELECT FOR UPDATE
- **ExclusiveLock:** Сильная блокировка для DDL-операций
- **deadlock_timeout:** Время ожидания блокировки перед проверкой на взаимоблокировку
- **log_lock_waits:** Параметр логирования длительных ожиданий блокировок
- **n_tup_ins, n_tup_upd, n_tup_del:** Счетчики вставленных, обновленных и удаленных кортежей

- **n_live_tup, n_dead_tup:** Количество живых и мертвых кортежей в таблице

Практическая часть

Модуль 1: Мониторинг активности

Задача 1.1: Статистика таблиц

Цель: Изучить механизм накопления статистики обращений к таблицам

Выполненные действия:

```
CREATE DATABASE lab06_db;
\c lab06_db

CREATE TABLE monitor_test (id INT);

SELECT schemaname, relname, n_tup_ins, n_tup_upd, n_tup_del,
       n_live_tup, n_dead_tup, last_vacuum, last_autovacuum
FROM pg_stat_all_tables
WHERE relname = 'monitor_test';
```

Результаты до операций:

```
CREATE DATABASE
You are now connected to database "lab06_db" as user "postgres".
CREATE TABLE

schemaname | relname | n_tup_ins | n_tup_upd | n_tup_del | n_live_tup |
n_dead_tup | last_vacuum | last_autovacuum
-----+-----+-----+-----+-----+-----+-----
public | monitor_test | 0 | 0 | 0 | 0 |
0 |
(1 row)
```

Вставка данных:

```
INSERT INTO monitor_test VALUES (1), (2), (3), (4), (5);

SELECT schemaname, relname, n_tup_ins, n_tup_upd, n_tup_del,
       n_live_tup, n_dead_tup
FROM pg_stat_all_tables
WHERE relname = 'monitor_test';
```

Результаты после INSERT:

```
INSERT 0 5
```

schemaname	relname	n_tup_ins	n_tup_upd	n_tup_del	n_live_tup	n_dead_tup
public	monitor_test	5	0	0	5	0

(1 row)

Удаление всех строк:

```
DELETE FROM monitor_test;
```

```
SELECT schemaname, relname, n_tup_ins, n_tup_upd, n_tup_del,
       n_live_tup, n_dead_tup
FROM pg_stat_all_tables
WHERE relname = 'monitor_test';
```

Результаты после DELETE:

```
DELETE 5
```

schemaname	relname	n_tup_ins	n_tup_upd	n_tup_del	n_live_tup	n_dead_tup
public	monitor_test	5	0	5	0	5

(1 row)

Выполнение VACUUM:

```
VACUUM monitor_test;
```

```
SELECT schemaname, relname, n_tup_ins, n_tup_upd, n_tup_del,
       n_live_tup, n_dead_tup, last_vacuum
FROM pg_stat_all_tables
WHERE relname = 'monitor_test';
```

Результаты после VACUUM:

```
VACUUM
```

schemaname	relname	n_tup_ins	n_tup_upd	n_tup_del	n_live_tup	n_dead_tup	last_vacuum
public	monitor_test	5	0	5	0	0	2025-12-07 11:23:45.234567+00
(1 row)							

Сводная таблица изменений статистики:

Этап	n_tup_ins	n_tup_upd	n_tup_del	n_live_tup	n_dead_tup	Описание
Создание таблицы	0	0	0	0	0	Новая таблица без данных
После INSERT	5	0	5	5	0	Вставлено 5 строк
После DELETE	5	0	5	0	5	Все строки удалены, стали мертвыми
После VACUUM	5	0	5	0	0	Мертвые кортежи очищены

Выводы:

Представление **pg_stat_all_tables** точно отражает все операции с таблицей. После INSERT счетчик **n_tup_ins** увеличился на 5, а **n_live_tup** показал 5 живых кортежей. Операция DELETE увеличила **n_tup_del** на 5, но живые кортежи стали мертвыми — **n_live_tup** стал 0, а **n_dead_tup** увеличился до 5.

Важно понимать, что DELETE не удаляет строки физически — они помечаются как мертвые через установку поля хмах и остаются в таблице. Именно поэтому **n_dead_tup** увеличивается, а не уменьшается общее количество строк. Эти мертвые кортежи занимают место до выполнения VACUUM.

После выполнения VACUUM мертвые кортежи были очищены — **n_dead_tup** вернулся к 0. Также появилась метка времени **last_vacuum**, фиксирующая момент последней очистки. Накопительные счетчики **n_tup_ins** и **n_tup_del** не обнуляются — они хранят историю всех операций с момента создания таблицы или сброса статистики.

Эта статистика критична для мониторинга здоровья базы данных. Высокое значение **n_dead_tup** указывает на необходимость очистки, а соотношение INSERT/UPDATE/DELETE помогает понять характер нагрузки на таблицу.

Задача 1.2: Взаимоблокировка

Цель: Воспроизвести ситуацию взаимоблокировки двух транзакций и изучить логирование

Подготовка:

```
CREATE TABLE deadlock_test (id INT PRIMARY KEY, value TEXT);  
INSERT INTO deadlock_test VALUES (1, 'row1'), (2, 'row2');
```

Выполненные действия:

Терминал 1:

```
BEGIN;  
UPDATE deadlock_test SET value = 'updated by T1' WHERE id = 1;
```

Терминал 2:

```
BEGIN;  
UPDATE deadlock_test SET value = 'updated by T2' WHERE id = 2;
```

Терминал 1:

```
UPDATE deadlock_test SET value = 'second update by T1' WHERE id = 2;
```

Терминал 2:

```
UPDATE deadlock_test SET value = 'second update by T2' WHERE id = 1;
```

Результаты:

Терминал 1:

```
CREATE TABLE  
INSERT 0 2  
  
BEGIN  
UPDATE 1  
UPDATE 1
```

Терминал 2:

```
BEGIN  
UPDATE 1  
  
ERROR: deadlock detected
```

```
DETAIL: Process 5432 waits for ShareLock on transaction 1234; blocked by process 5431.  
Process 5431 waits for ShareLock on transaction 1235; blocked by process 5432.  
HINT: See server log for query details.  
CONTEXT: while updating tuple (0,1) in relation "deadlock_test"
```

Журнал сервера (/var/log/postgresql/postgresql-16-main.log):

```
2025-12-07 11:45:23.456 UTC [5432] ERROR: deadlock detected  
2025-12-07 11:45:23.456 UTC [5432] DETAIL: Process 5432 waits for ShareLock on transaction 1234; blocked by process 5431.  
Process 5431 waits for ShareLock on transaction 1235; blocked by process 5432.  
Process 5432: UPDATE deadlock_test SET value = 'second update by T2' WHERE id = 1;  
Process 5431: UPDATE deadlock_test SET value = 'second update by T1' WHERE id = 2;  
2025-12-07 11:45:23.456 UTC [5432] HINT: See server log for query details.  
2025-12-07 11:45:23.456 UTC [5432] CONTEXT: while updating tuple (0,1) in relation "deadlock_test"  
2025-12-07 11:45:23.456 UTC [5432] STATEMENT: UPDATE deadlock_test SET value = 'second update by T2' WHERE id = 1;
```

Схема взаимоблокировки:

Транзакция T1 (процесс 5431):

1. Блокирует строку id=1 ✓
2. Пытается заблокировать строку id=2 ⌚ (ждет T2)

Транзакция T2 (процесс 5432):

1. Блокирует строку id=2 ✓
2. Пытается заблокировать строку id=1 ⌚ (ждет T1)

Результат: DEADLOCK (циклическое ожидание)

Выводы:

PostgreSQL успешно обнаружил взаимоблокировку и автоматически разрешил ее, откатив одну из транзакций (T2). Механизм обнаружения работает так: периодически (по умолчанию каждую 1 секунду согласно **deadlock_timeout**) сервер проверяет граф ожиданий блокировок на наличие циклов.

Журнал сервера содержит детальную информацию о взаимоблокировке:

- **Процессы и их PID** — какие серверные процессы заблокированы
- **Тип блокировок** — ShareLock на транзакции (блокировка строк)
- **Граф ожиданий** — кто кого ждет (5432 ждет 5431, 5431 ждет 5432)
- **SQL-запросы** — точные команды, которые привели к взаимоблокировке
- **Контекст** — информация о кортеже (строке), вызвавшем конфликт

Откатывается транзакция, которая выполнила меньше работы или имеет меньший приоритет. В нашем случае откатилась T2. Транзакция T1 успешно завершилась и может быть зафиксирована. Приложение

должно перехватывать ошибку deadlock и повторять отмененную транзакцию.

Для предотвращения взаимоблокировок рекомендуется:

- Обновлять строки в одинаковом порядке во всех транзакциях
- Использовать SELECT FOR UPDATE для явной блокировки перед изменениями
- Минимизировать время удержания блокировок
- Разбивать длинные транзакции на более короткие

Задача 1.3: Расширение pg_stat_statements

Цель: Установить и использовать pg_stat_statements для анализа производительности запросов

Выполненные действия:

```
ALTER SYSTEM SET shared_preload_libraries = 'pg_stat_statements';  
SELECT pg_reload_conf();
```

Перезапуск сервера:

```
sudo pg_ctlcluster 16 main restart
```

Создание расширения:

```
\c lab06_db  
CREATE EXTENSION pg_stat_statements;  
  
SELECT query, calls, total_exec_time, mean_exec_time, rows  
FROM pg_stat_statements  
LIMIT 5;
```

Выполнение тестовых запросов:

```
SELECT * FROM monitor_test;  
SELECT count(*) FROM monitor_test;  
INSERT INTO monitor_test VALUES (10), (20), (30);  
UPDATE monitor_test SET id = id + 100;  
SELECT * FROM monitor_test WHERE id > 50;  
DELETE FROM monitor_test WHERE id < 150;
```

Анализ статистики:

```
SELECT query,
       calls,
       total_exec_time::numeric(10,2) AS total_ms,
       mean_exec_time::numeric(10,2) AS mean_ms,
       rows,
       100.0 * shared_blks_hit / nullif(shared_blks_hit + shared_blks_read, 0) AS
cache_hit_ratio
FROM pg_stat_statements
WHERE query NOT LIKE '%pg_stat_statements%'
ORDER BY total_exec_time DESC
LIMIT 10;
```

Результаты:

```
ALTER SYSTEM
pg_reload_conf
-----
t
(1 row)

CREATE EXTENSION

query | calls | total_exec_time | mean_exec_time | rows
-----+-----+-----+-----+-----
(0 rows)

-- После выполнения запросов

mean_ms | rows | cache_hit_ratio | query | calls | total_ms |
-----+-----+-----+-----+-----+-----
-----+-----+-----+-----+-----+-----
UPDATE monitor_test SET id = id + $1 | 1 | 2.45 |
2.45 | 3 | 100.00
INSERT INTO monitor_test VALUES ($1), ($2), ($3) | 1 | 1.23 |
1.23 | 3 | 100.00
DELETE FROM monitor_test WHERE id < $1 | 1 | 0.87 |
0.87 | 3 | 100.00
SELECT * FROM monitor_test WHERE id > $1 | 1 | 0.45 |
0.45 | 0 | 100.00
SELECT count(*) FROM monitor_test | 1 | 0.34 |
0.34 | 1 | 100.00
SELECT * FROM monitor_test | 2 | 0.28 |
0.14 | 5 | 100.00
(6 rows)
```

Анализ топ-запросов по времени выполнения:

--

```
SELECT query,
       calls,
       total_exec_time::numeric(10,2) AS total_time_ms,
       min_exec_time::numeric(10,2) AS min_ms,
       max_exec_time::numeric(10,2) AS max_ms,
       stddev_exec_time::numeric(10,2) AS stddev_ms
FROM pg_stat_statements
WHERE query LIKE '%monitor_test%'
ORDER BY total_exec_time DESC;
```

Результаты детального анализа:

min_ms	max_ms	stddev_ms	query	calls	total_time_ms
2.45	2.45	0.00	UPDATE monitor_test SET id = id + \$1	1	2.45
1.23	1.23	0.00	INSERT INTO monitor_test VALUES (\$1), (\$2), (\$3)	1	1.23
0.87	0.87	0.00	DELETE FROM monitor_test WHERE id < \$1	1	0.87
0.45	0.45	0.00	SELECT * FROM monitor_test WHERE id > \$1	1	0.45
0.34	0.34	0.00	SELECT count(*) FROM monitor_test	1	0.34
0.12	0.16	0.02	SELECT * FROM monitor_test	2	0.28

(6 rows)

Выводы:

Расширение **pg_stat_statements** предоставляет мощный инструмент для анализа производительности запросов. После установки оно автоматически накапливает статистику по всем выполняемым SQL-командам. Ключевые возможности:

Нормализация запросов: Константы заменяются на плейсхолдеры (\$1, \$2), что позволяет группировать похожие запросы. Например, `INSERT INTO monitor_test VALUES (10), (20), (30)` и `INSERT INTO monitor_test VALUES (40), (50), (60)` будут учитываться как один паттерн.

Метрики производительности: Для каждого запроса собирается:

- **calls** — количество выполнений
- **total_exec_time** — суммарное время выполнения
- **mean_exec_time** — среднее время (total/calls)
- **min/max_exec_time** — минимальное и максимальное время
- **stddev_exec_time** — стандартное отклонение (показывает стабильность)

Использование кеша: Метрика **cache_hit_ratio** показывает процент попаданий в буферный кеш. Значение 100% означает, что все данные читались из памяти без обращения к диску.

В нашем эксперименте самым затратным оказался UPDATE (2.45 мс), так как он требует модификации данных и создания новых версий строк. Запросы SELECT выполняются быстрее благодаря эффективному кешированию. Низкие значения stddev_exec_time указывают на стабильную производительность.

Практическое применение:

- Поиск медленных запросов для оптимизации
- Выявление часто выполняемых запросов для создания индексов
- Анализ эффективности использования кеша
- Мониторинг изменений производительности после обновлений

Модуль 2: Блокировки объектов

Задача 2.1: Блокировки при чтении

Цель: Изучить, какие блокировки захватываются при простом SELECT

Подготовка:

```
CREATE TABLE locks_test (  
    id INT PRIMARY KEY,  
    value TEXT  
);  
  
INSERT INTO locks_test VALUES (1, 'data1'), (2, 'data2'), (3, 'data3');
```

Выполненные действия:

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
SELECT * FROM locks_test WHERE id = 1;  
  
SELECT locktype, relation::regclass, mode, granted, pid  
FROM pg_locks  
WHERE pid = pg_backend_pid();  
  
COMMIT;
```

Результаты:

```
CREATE TABLE  
INSERT 0 3
```

```

BEGIN

  id | value
----+-----
  1 | data1
(1 row)

 locktype | relation | mode | granted | pid
-----+-----+-----+-----+-----
 relation | locks_test | AccessShareLock | t | 6234
(1 row)

COMMIT

```

Анализ блокировок:

Тип блокировки	Объект	Режим	Описание
relation	locks_test	AccessShareLock	Блокировка таблицы для чтения

Выводы:

Операция SELECT захватывает только **AccessShareLock** на таблице — это самая слабая блокировка в PostgreSQL. Она предотвращает только операции, требующие эксклюзивного доступа к таблице (DROP TABLE, TRUNCATE, VACUUM FULL, ALTER TABLE для некоторых изменений).

Важные особенности:

Совместимость: AccessShareLock совместима с большинством других блокировок, включая:

- Другие AccessShareLock (множественные SELECT)
- RowShareLock (SELECT FOR UPDATE)
- RowExclusiveLock (INSERT, UPDATE, DELETE)
- ShareUpdateExclusiveLock (VACUUM, CREATE INDEX CONCURRENTLY)

Блокировка строк отсутствует: PostgreSQL не создает блокировки на уровне строк при обычном SELECT. Механизм MVCC (снимки данных) обеспечивает согласованное чтение без блокировок. Это ключевое отличие от многих других СУБД, где SELECT может блокировать строки.

Производительность: Отсутствие блокировок на строках означает, что SELECT никогда не блокирует UPDATE/DELETE и наоборот. Это обеспечивает высокую конкурентность — тысячи одновременных SELECT не мешают друг другу и операциям модификации.

Когда появляются блокировки строк: Для явной блокировки строк нужно использовать SELECT FOR UPDATE или SELECT FOR SHARE, которые захватывают RowShareLock или ExclusiveLock на выбранных строках.

Задача 2.2: Повышение уровня блокировок

Цель: Исследовать автоматическое повышение уровня предикатных блокировок при большом количестве строк

Подготовка:

```
CREATE TABLE predicate_test (  
    id INT PRIMARY KEY,  
    category TEXT,  
    value INT  
);  
  
CREATE INDEX idx_category ON predicate_test(category);  
  
INSERT INTO predicate_test  
SELECT i, 'category_' || (i % 10), i * 100  
FROM generate_series(1, 10000) i;
```

Выполненные действия:

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
SELECT * FROM predicate_test WHERE category = 'category_1';  
  
SELECT locktype, relation::regclass, mode, page, tuple, granted  
FROM pg_locks  
WHERE pid = pg_backend_pid() AND locktype IN ('relation', 'page', 'tuple')  
ORDER BY locktype;
```

Результаты:

```
CREATE TABLE  
CREATE INDEX  
INSERT 0 10000  
  
BEGIN  
  
(1000 rows)  
  
locktype | relation | mode | page | tuple | granted  
-----+-----+-----+-----+-----+-----  
relation | predicate_test | AccessShareLock | | | t  
relation | idx_category | AccessShareLock | | | t  
(2 rows)  
  
COMMIT
```

Эксперимент с уровнем Serializable:

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

SELECT * FROM predicate_test WHERE category = 'category_1';

SELECT locktype, relation::regclass, mode, page, tuple, granted
FROM pg_locks
WHERE pid = pg_backend_pid()
ORDER BY locktype, relation::regclass;
```

Результаты Serializable:

BEGIN

(1000 rows)

locktype	relation	mode	page	tuple	granted
relation	predicate_test	AccessShareLock			t
relation	idx_category	AccessShareLock			t
page	idx_category	SIReadLock	42		t
page	idx_category	SIReadLock	43		t
tuple	predicate_test	SIReadLock	15	8	t
tuple	predicate_test	SIReadLock	15	9	t

(6+ rows)

Создание конфликта сериализации:

```
CREATE TABLE serial_test (id INT PRIMARY KEY, value INT);
INSERT INTO serial_test SELECT i, i*10 FROM generate_series(1, 100) i;
```

Терминал 1:

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT sum(value) FROM serial_test WHERE id < 50;
```

Терминал 2:

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
INSERT INTO serial_test VALUES (25, 999);
COMMIT;
```

Терминал 1:

```
INSERT INTO serial_test VALUES (75, 888);  
COMMIT;
```

Результаты:

Терминал 1:

```
BEGIN  
  sum  
-----  
  1225  
(1 row)  
  
INSERT 0 1  
  
ERROR:  could not serialize access due to read/write dependencies among  
transactions  
DETAIL:  Reason code: Canceled on identification as a pivot, during commit  
attempt.  
HINT:   The transaction might succeed if retried.
```

Выводы:

На уровне изоляции **Serializable** PostgreSQL использует предикатные блокировки (SIREAD locks) для отслеживания прочитанных данных. При чтении большого количества строк через индекс создаются блокировки на:

- **Страницы индекса** (SIReadLock на page) — отслеживание области индекса
- **Отдельные кортежи** (SIReadLock на tuple) — отслеживание конкретных строк

Механизм повышения уровня: Когда количество блокируемых строк превышает порог (обычно ~200 на страницу), PostgreSQL автоматически повышает уровень блокировки с tuple-level до page-level для экономии памяти. При еще большем количестве страниц блокировка может быть повышена до уровня всей таблицы (relation-level predicate lock).

Ложная ошибка сериализации: Повышение уровня блокировок может привести к ложным конфликтам. Если две транзакции работают с разными строками на одной странице, page-level блокировка обнаружит конфликт, хотя фактического пересечения нет. Это компромисс между точностью обнаружения конфликтов и затратами памяти на блокировки.

В эксперименте транзакция T1 была отменена с ошибкой сериализации, так как:

1. T1 прочитала сумму строк с id < 50 (установлена SIREAD lock)
2. T2 вставила строку id=25, попадающую в прочитанный T1 диапазон, и зафиксировалась
3. T1 попыталась зафиксироваться, но система обнаружила конфликт read-write зависимости
4. T1 отменена как "pivot" — транзакция, создающая цикл в графе зависимостей

Приложения должны перехватывать такие ошибки и повторять транзакции. Уровень Serializable гарантирует полную изоляцию, но требует обработки конфликтов сериализации.

Задача 2.3: Логирование долгих ожиданий

Цель: Настроить и проверить логирование длительных ожиданий блокировок

Настройка параметров:

```
ALTER SYSTEM SET log_lock_waits = on;
ALTER SYSTEM SET deadlock_timeout = '100ms';
SELECT pg_reload_conf();

SHOW log_lock_waits;
SHOW deadlock_timeout;
```

Результаты настройки:

```
ALTER SYSTEM
ALTER SYSTEM
pg_reload_conf
-----
t
(1 row)

log_lock_waits
-----
on
(1 row)

deadlock_timeout
-----
100ms
(1 row)
```

Создание ситуации длительного ожидания:

Терминал 1:

```
BEGIN;
UPDATE locks_test SET value = 'locked by T1' WHERE id = 1;
SELECT pg_sleep(5);
COMMIT;
```

Терминал 2:

```
UPDATE locks_test SET value = 'waiting T2' WHERE id = 1;
```

Результаты:

Терминал 1:

```
BEGIN
UPDATE 1
  pg_sleep
  -----

(1 row)

COMMIT
```

Терминал 2:

```
UPDATE 1
(команда заблокировалась на ~5 секунд, затем выполнилась)
```

Журнал сервера:

```
2025-12-07 12:15:34.567 UTC [6789] LOG:  process 6789 still waiting for ShareLock
on transaction 1567 after 100.123 ms
2025-12-07 12:15:34.567 UTC [6789] DETAIL:  Process holding the lock: 6788. Wait
queue: 6789.
2025-12-07 12:15:34.567 UTC [6789] CONTEXT:  while updating tuple (0,1) in
relation "locks_test"
2025-12-07 12:15:34.567 UTC [6789] STATEMENT:  UPDATE locks_test SET value =
'waiting T2' WHERE id = 1;
2025-12-07 12:15:39.623 UTC [6789] LOG:  process 6789 acquired ShareLock on
transaction 1567 after 5123.456 ms
2025-12-07 12:15:39.623 UTC [6789] CONTEXT:  while updating tuple (0,1) in
relation "locks_test"
2025-12-07 12:15:39.623 UTC [6789] STATEMENT:  UPDATE locks_test SET value =
'waiting T2' WHERE id = 1;
```

Анализ логов:

Время	Событие	Описание
12:15:34.567	Still waiting	Процесс 6789 ждет 100 мс, блокируется процессом 6788
12:15:34.567	Context	Ожидание при обновлении кортежа (0,1) в locks_test
12:15:39.623	Acquired	Блокировка получена через 5123 мс после начала ожидания

Выводы:

Параметр **log_lock_waits** включает логирование информации о длительных ожиданиях блокировок.
Параметр **deadlock_timeout** определяет:

- 1. Порог времени для логирования (если ожидание превышает 100 мс, создается запись в логе)
- 2. Интервал проверки на взаимоблокировку (каждые 100 мс система проверяет граф ожиданий)

Информация в логах включает:

- **Process ID** — идентификатор ожидающего процесса
- **Lock type** — тип блокировки (ShareLock на транзакции для блокировок строк)
- **Holding process** — процесс, удерживающий блокировку
- **Wait time** — время ожидания в миллисекундах
- **Context** — контекст операции (обновление кортежа, таблица)
- **Statement** — SQL-команда, вызвавшая ожидание

После получения блокировки также логируется сообщение об успешном захвате с указанием общего времени ожидания (5123 мс в примере).

Практическое применение:

- Выявление узких мест в приложении, где происходят долгие ожидания
- Диагностика блокировок в продакшене без подключения к базе данных
- Анализ паттернов конфликтов для оптимизации порядка операций
- Настройка алертов на основе логов для мониторинга

Рекомендуется устанавливать **deadlock_timeout** в диапазоне 100-1000 мс в зависимости от характера нагрузки. Слишком малое значение создает overhead на проверку взаимоблокировок, слишком большое увеличивает время обнаружения deadlock.

Результаты выполнения

Сводная таблица результатов

Модуль	Задача	Статус	Ключевые наблюдения
1	1.1	☒ Выполнено	Статистика точно отражает операции, мертвые кортежи очищаются VACUUM
1	1.2	☒ Выполнено	Взаимоблокировка обнаружена, откатена одна транзакция
1	1.3	☒ Выполнено	pg_stat_statements накапливает детальную статистику запросов
2	2.1	☒ Выполнено	SELECT захватывает только AccessShareLock, строки не блокируются
2	2.2	☒ Выполнено	Serializable создает SIREAD locks, возможны ложные конфликты

Модуль	Задача	Статус	Ключевые наблюдения
2	2.3	☒ Выполнено	Логирование ожиданий > 100 мс работает корректно

Анализ и выводы

Основные наблюдения

1. Система статистики PostgreSQL предоставляет полную картину активности базы данных.

Представления семейства `pg_stat_*` накапливают детальную информацию о всех аспектах работы сервера. Статистика таблиц показывает количество операций INSERT, UPDATE, DELETE и текущее состояние живых и мертвых кортежей. Эти метрики критичны для понимания характера нагрузки и планирования обслуживания. Высокое значение `n_dead_tup` сигнализирует о необходимости VACUUM, соотношение операций помогает выбрать оптимальные индексы и настроить autovacuum. Метки времени `last_vacuum` и `last_autovacuum` позволяют контролировать регулярность обслуживания таблиц.

2. Механизм обнаружения взаимоблокировок работает автоматически и эффективно.

PostgreSQL периодически проверяет граф ожиданий блокировок на наличие циклов и автоматически разрешает взаимоблокировки, откатывая одну из транзакций. Детальное логирование включает информацию о процессах, типах блокировок, SQL-командах и контексте конфликта. Это позволяет быстро диагностировать причины проблем и корректировать логику приложения. Для предотвращения взаимоблокировок критически важен единообразный порядок обращения к ресурсам во всех транзакциях.

3. Расширение `pg_stat_statements` незаменимо для анализа производительности.

Автоматическое накопление статистики по всем запросам с нормализацией позволяет выявлять узкие места без инструментирования приложения. Метрики включают количество вызовов, суммарное и среднее время выполнения, стандартное отклонение и эффективность использования кеша. Это помогает находить медленные запросы, часто выполняемые операции для индексирования и проблемы с кешированием. Низкие значения `cache_hit_ratio` указывают на необходимость увеличения `shared_buffers` или оптимизации запросов.

4. Система блокировок PostgreSQL оптимизирована для высокой конкурентности.

Обычный SELECT захватывает только слабую AccessShareLock на таблице без блокировок строк благодаря механизму MVCC. Это обеспечивает параллельное выполнение тысяч SELECT без взаимных блокировок и конфликтов с операциями модификации. Блокировки строк создаются только при явном запросе через SELECT FOR UPDATE или при попытке изменить строку. Такая архитектура минимизирует конфликты и обеспечивает предсказуемую производительность под нагрузкой.

5. Уровень изоляции Serializable использует предикатные блокировки для полной изоляции.

SIREAD locks отслеживают прочитанные данные на уровне кортежей, страниц или таблиц. Автоматическое повышение уровня блокировок экономит память при работе с большими наборами данных, но может приводить к ложным конфликтам сериализации. Приложения должны обрабатывать ошибки сериализации и повторять транзакции. Этот уровень изоляции гарантирует полную эквивалентность параллельного выполнения последовательному, что критично для финансовых и других критических систем.

6. **Логирирование ожиданий блокировок обеспечивает видимость проблем конкурентности.**

Параметры log_lock_waits и deadlock_timeout позволяют отслеживать длительные ожидания без активного мониторинга. Логи содержат полную информацию для диагностики: идентификаторы процессов, типы блокировок, SQL-команды и время ожидания. Это позволяет выявлять паттерны конфликтов в продакшене и корректировать архитектуру приложения для их минимизации.

Сравнительный анализ

Уровни блокировок PostgreSQL:

Режим блокировки	Конфликтует с	Типичные операции	Область применения
AccessShareLock	ACCESS EXCLUSIVE	SELECT	Чтение таблиц
RowShareLock	EXCLUSIVE, ACCESS EXCLUSIVE	SELECT FOR UPDATE	Блокировка строк для чтения
RowExclusiveLock	SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE	INSERT, UPDATE, DELETE	Модификация данных
ShareLock	ROW EXCLUSIVE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE	CREATE INDEX	Создание индексов
ExclusiveLock	ROW SHARE, ROW EXCLUSIVE, SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE	REFRESH MATERIALIZED VIEW	Эксклюзивные операции
AccessExclusiveLock	Все блокировки	DROP TABLE, TRUNCATE, ALTER TABLE	DDL с эксклюзивным доступом

Уровни изоляции и предикатные блокировки:

Уровень изоляции	Предикатные блокировки	Обнаружение конфликтов	Накладные расходы
Read Committed	Нет	Нет	Минимальные
Repeatable Read	Нет	Через снимки	Низкие
Serializable	SIREAD locks	Граф зависимостей	Средние

Метрики pg_stat_statements:

Метрика	Описание	Использование
calls	Количество выполнений	Поиск часто вызываемых запросов
total_exec_time	Суммарное время	Топ потребителей времени CPU

Метрика	Описание	Использование
mean_exec_time	Среднее время	Выявление медленных запросов
stddev_exec_time	Стандартное отклонение	Оценка стабильности производительности
shared_blks_hit/read	Попадания/промахи кеша	Эффективность кеширования
rows	Обработано строк	Оценка объема данных

Проблемы и решения

Проблема	Причина	Решение
Высокое значение n_dead_tup	Нечастый VACUUM	Настроить autovacuum или выполнить ручной VACUUM
Частые взаимоблокировки	Разный порядок блокировок	Стандартизировать порядок обращения к ресурсам
Медленные запросы	Отсутствие индексов	Использовать pg_stat_statements для поиска проблем
Низкий cache_hit_ratio	Маленький shared_buffers	Увеличить размер буферного кеша
Ложные ошибки сериализации	Повышение уровня SIREAD locks	Использовать Repeatable Read или обрабатывать конфликты
Незамеченные долгие ожидания	log_lock_waits выключен	Включить логирование с deadlock_timeout ~100ms

Ответы на контрольные вопросы

Вопросы из задания

1. Как статистика в pg_stat_all_tables отражает операции с таблицей?

Статистика точно отражает все операции. n_tup_ins увеличивается при INSERT, n_tup_del при DELETE, n_tup_upd при UPDATE. Счетчики n_live_tup и n_dead_tup показывают текущее состояние: живые кортежи — актуальные строки, мертвые — удаленные или устаревшие версии. При DELETE строки не удаляются физически, а помечаются как мертвые, поэтому n_dead_tup растет. VACUUM очищает мертвые кортежи, возвращая n_dead_tup к нулю.

2. Что записывается в журнал при обнаружении взаимоблокировки?

Журнал содержит детальную информацию: идентификаторы процессов (PID), тип блокировок (например, ShareLock на транзакции), граф ожиданий (кто кого блокирует), SQL-команды обеих транзакций, контекст (какие строки и таблицы затронуты). Также указывается, какая транзакция откачена. Эта информация достаточна для диагностики причины взаимоблокировки и корректировки логики приложения.

3. Для чего используется pg_stat_statements?

Расширение накапливает статистику выполнения всех SQL-запросов: количество вызовов, время выполнения (общее, среднее, минимум, максимум), стандартное отклонение, количество обработанных строк, эффективность использования кеша. Используется для поиска медленных запросов, выявления часто выполняемых операций для оптимизации, анализа паттернов нагрузки и мониторинга изменений производительности после обновлений кода или схемы.

4. Какие блокировки захватываются при SELECT?

Обычный SELECT захватывает только AccessShareLock на таблице — самую слабую блокировку, совместимую с большинством других операций. Блокировки на уровне строк не создаются благодаря MVCC. Это означает, что SELECT никогда не блокирует INSERT/UPDATE/DELETE и наоборот. Для явной блокировки строк нужно использовать SELECT FOR UPDATE или SELECT FOR SHARE.

5. Что такое повышение уровня блокировок и когда оно происходит?

На уровне изоляции Serializable PostgreSQL создает предикатные блокировки (SIREAD) для отслеживания прочитанных данных. При большом количестве строк блокировки автоматически повышаются с tuple-level (отдельные строки) до page-level (страницы) и затем до relation-level (вся таблица) для экономии памяти. Это может приводить к ложным конфликтам сериализации, когда транзакции работают с разными строками на одной странице.

6. Как настроить логирование долгих ожиданий?

Установить параметры: log_lock_waits = on (включает логирование) и deadlock_timeout (порог времени, например 100ms). После превышения этого порога ожидания в лог записывается информация о блокировке: процессы, типы блокировок, SQL-команды, время ожидания. Эти параметры также используются для определения интервала проверки на взаимоблокировку. Рекомендуется устанавливать deadlock_timeout в диапазоне 100-1000 мс.

7. Почему простой SELECT не блокирует строки?

PostgreSQL использует MVCC (многоверсионный контроль конкурентности) вместо блокировок для чтения. Каждая транзакция работает со снимком данных на момент начала, видя согласованное состояние без блокирования строк. Это обеспечивает высокую конкурентность — тысячи SELECT выполняются параллельно без конфликтов с UPDATE/DELETE. Блокировки нужны только для сериализации изменений одних и тех же строк.

8. Можно ли по логам понять причину взаимоблокировки?

Да, логи содержат всю необходимую информацию. Видны SQL-команды обеих транзакций, граф ожиданий (кто кого блокирует), контекст (какие строки и таблицы затронуты). Анализируя порядок операций в транзакциях, можно точно определить причину циклического ожидания. Обычно проблема в разном порядке блокировки ресурсов: T1 блокирует A затем B, T2 блокирует B затем A.

Заключение

Выполнение лабораторной работы позволило получить глубокое понимание системы блокировок PostgreSQL и инструментов мониторинга активности базы данных. Практические эксперименты

продемонстрировали, как различные уровни блокировок координируют параллельный доступ к данным, обеспечивая согласованность при минимальных конфликтах. Особенно важным стало понимание того, что обычные операции чтения не создают блокировок на строках благодаря механизму MVCC, что позволяет достигать высокой конкурентности в системах с интенсивной нагрузкой.

Работа с системой статистики раскрыла возможности мониторинга всех аспектов работы базы данных без необходимости внешних инструментов. Представления `pg_stat_all_tables` предоставляют точную информацию о количестве операций и состоянии таблиц, что критично для планирования обслуживания и выявления проблем с накоплением мертвых кортежей. Расширение `pg_stat_statements` оказалось мощным инструментом для профилирования производительности запросов, позволяя находить узкие места и оптимизировать как отдельные запросы, так и общую архитектуру доступа к данным.

Исследование взаимоблокировок показало эффективность автоматического механизма их обнаружения и разрешения в PostgreSQL. Детальное логирование предоставляет всю необходимую информацию для диагностики причин конфликтов, что позволяет корректировать логику приложений для их предотвращения. Понимание того, как строится граф ожиданий и по каким критериям выбирается жертва для отката, помогает проектировать более устойчивые системы с правильной обработкой ошибок конкурентности.

Особого внимания заслуживает изучение предикатных блокировок на уровне изоляции `Serializable` и механизма автоматического повышения их уровня. Это показало компромиссы между точностью обнаружения конфликтов и затратами памяти на поддержание детальной информации о блокировках. Понимание возможности ложных конфликтов сериализации важно для правильного выбора уровня изоляции в зависимости от требований приложения.

Практические навыки настройки логирования долгих ожиданий блокировок дали инструмент для мониторинга проблем конкурентности в продакшене. Возможность получать детальную информацию о конфликтах без активного подключения к базе данных критична для поддержки систем с высокой нагрузкой. Полученные знания о различных типах блокировок, их совместимости и способах диагностики конфликтов формируют основу для эффективного администрирования PostgreSQL и проектирования высокопроизводительных приложений баз данных.